

Python E-Course

Module 8 — Error Handling

Detailed Notes

Module Overview

Level: Intermediate

Duration: ~1 week

Prerequisites: Modules 1-7

What You'll Learn

- Catch and handle exceptions with try/except.
- Use else and finally clauses.
- Raise exceptions deliberately.
- Define custom exception classes (preview of OOP).
- Debug effectively when things break.

Lessons

#	Lesson	Duration
1	try / except Basics	30 min
2	The Exception Hierarchy	25 min
3	raise, else, finally	30 min
4	Custom Exceptions	25 min
5	Debugging Techniques	30 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 9: Object-Oriented Programming.

Lesson 1: try / except Basics

Module: 8 — Error Handling

Duration: 30 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Wrap risky code in try/except to handle errors gracefully.
- Catch specific exception types.
- Access the exception object via as.

Prerequisites

- Modules 1-7, especially file I/O which raises common exceptions.

1. Why This Matters

Until now, any error crashed your program. Real programs catch errors, recover, log, or report cleanly. try/except is how Python lets you say "this might fail — here's what to do if it does."

2. Concept

2.1 Basic syntax

```
try:
    risky_code()
except ExceptionType:
    handle_it()

try:
    n = int(input("Number: "))
    print(10 / n)
except ValueError:
    print("That wasn't a number.")
except ZeroDivisionError:
    print("Can't divide by zero.")
```

2.2 Multiple except blocks

Try each in order; the first matching type runs.

2.3 Catching multiple types at once

```
try:
    risky()
except (ValueError, TypeError):
    print("Bad input")
```

2.4 Catching the exception object

```
try:
    int("hello")
```

```
except ValueError as e:
    print("Failed:", e)
```

`e` is the exception object — you can read its message, attributes, etc.

2.5 Catch-all (use sparingly)

```
try:
    risky()
except Exception as e:
    print("Something went wrong:", e)
```

`except Exception` catches anything except `KeyboardInterrupt` and `SystemExit`. Don't use bare `except:` — it hides `Ctrl+C` and shutdown signals.

2.6 Don't swallow errors silently

Bad pattern:

```
try:
    risky()
except Exception:
    pass    # silent failure – bugs disappear
```

If you must ignore, at least log it. Better: only catch what you can actually handle.

3. Code Examples

Example 1: Handle bad input

```
raw = input("Number: ")
try:
    n = int(raw)
    print("Doubled:", n * 2)
except ValueError:
    print(f"'{raw}' is not a valid number.")
```

Sample:

```
Number: 5
Doubled: 10

Number: hello
'hello' is not a valid number.
```

Example 2: File not found

```
try:
    with open("missing.txt") as f:
        print(f.read())
except FileNotFoundError:
    print("File doesn't exist – creating it.")
    open("missing.txt", "w").write("created\n")
```

Expected Output: File doesn't exist — creating it.

Example 3: Multiple exception types

```
def safe_div(a, b):
    try:
        return a / b
    except (ZeroDivisionError, TypeError) as e:
        print("Error:", type(e).__name__, e)
        return None

print(safe_div(10, 0))
print(safe_div(10, "2"))
print(safe_div(10, 2))
```

Expected Output:

```
Error: ZeroDivisionError division by zero
None
Error: TypeError unsupported operand type(s) for /: 'int' and 'str'
None
5.0
```

Example 4: Retry loop on bad input

```
while True:
    try:
        age = int(input("Age: "))
        break
    except ValueError:
        print("Please enter a whole number.")
print(f"You're {age}.")
```

Sample:

```
Age: twenty
Please enter a whole number.
Age: 20
You're 20.
```

4. Common Mistakes

Mistake	What Happens	Fix
Bare except:	Hides Ctrl+C and bugs	Use except SpecificError or except Exception
Catching too broadly	Hides real bugs	Catch the narrowest type that's actually expected
try block too large	Hard to know which line failed	Wrap only the risky line
Silently ignoring	Bugs vanish into thin air	At minimum, log the exception

5. Practice Tasks

- Ask for a number; catch the ValueError and re-prompt.
- Open a non-existent file; catch FileNotFoundError and print a friendly message.

- Wrap `10 / x` in `try/except`; handle both `ZeroDivisionError` and `TypeError`.

6. Key Takeaways

- `try/except` lets you handle anticipated failures.
- Catch the specific exception you expect.
- Use `e` to access the exception object.
- Avoid bare `except:` and silent swallowing.

7. What's Next

Tour the exception hierarchy so you know what exists and which to catch.

Lesson 2: The Exception Hierarchy

Module: 8 — Error Handling

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Recognize the major branches of Python's built-in exception tree.
- Catch broad parent classes when appropriate.
- Identify the most common exceptions you'll meet.

Prerequisites

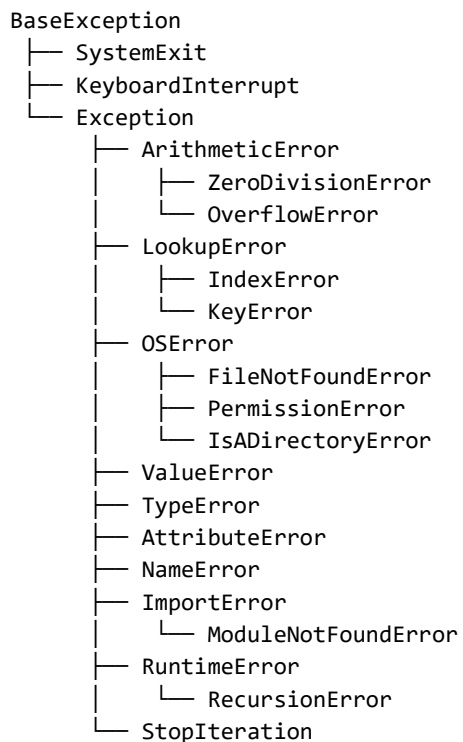
- Module 8 Lesson 1

1. Why This Matters

Catching Exception is a sledgehammer. Catching ValueError is a scalpel. Knowing the tree lets you decide which class is the right "level" to catch — broad enough to be useful, narrow enough not to mask bugs.

2. Concept

2.1 Top of the tree



Exception is the parent of most things you'd catch. BaseException includes the "we want to crash" cases (KeyboardInterrupt, SystemExit) — don't catch those.

2.2 Catching a parent catches its children

```
try:
    [1, 2, 3][99]      # IndexError
except LookupError:
    print("missing")   # catches both IndexError and KeyError
```

2.3 Common exceptions you'll meet

Exception	Why it fires
ValueError	Right type, wrong value: int("abc")
TypeError	Wrong type: len(5), "a" + 1
KeyError	Missing dict key: d["x"]
IndexError	Index out of range: lst[99]
AttributeError	Missing method/attribute: "hi".foo()
FileNotFoundError	Missing file in open("r")
PermissionError	OS denied access
ZeroDivisionError	x / 0
ImportError / ModuleNotFoundError	import nope
StopIteration	End of an iterator (rarely caught manually)
RecursionError	Too deep recursion

2.4 Inspecting an exception

```
try:
    int("abc")
except Exception as e:
    print(type(e).__name__)
    print(e.args)
```

2.5 Print the traceback

```
import traceback
try:
    1 / 0
except Exception:
    traceback.print_exc()    # shows the full traceback
```

Useful when logging an error you still want to know about.

3. Code Examples

Example 1: Right type, wrong value

```
try:
    n = int("twenty")
except ValueError as e:
    print("Bad value:", e)
```

Expected Output: Bad value: invalid literal for int() with base 10: 'twenty'

Example 2: Wrong type

```
try:
    result = "a" + 5
except TypeError as e:
    print("Bad type:", e)
```

Expected Output: Bad type: can only concatenate str (not "int") to str

Example 3: Catching a parent class

```
try:
    {"a": 1}["b"]
except LookupError:
    print("missing key or index")
```

Expected Output: missing key or index

Example 4: Inspecting

```
try:
    open("nope.txt")
except OSError as e:
    print("errno:", e.errno)
    print("msg:", e.strerror)
    print("filename:", e.filename)
```

Expected Output (paths/errno may vary):

```
errno: 2
msg: No such file or directory
filename: nope.txt
```

4. Common Mistakes

Mistake	What Happens	Fix
Catching BaseException	Catches Ctrl+C	Use Exception (or narrower)
Catching Exception everywhere	Hides bugs	Catch the narrowest type useful
Catching child after parent	Child never runs	Order: narrowest first
Catching what you don't recover from	Pretends success	Let it crash if you can't fix it

5. Practice Tasks

- Trigger and catch a KeyError, an IndexError, and a ValueError. Print the exception class name for each.
- Use one except LookupError to catch both KeyError and IndexError and confirm it works.
- Try open("nope") and read e.errno, e.strerror, e.filename.

6. Key Takeaways

- Python's exceptions are a tree rooted at BaseException.
- Catch Exception (or narrower) — never BaseException directly.

- Parents catch their children — useful for grouping.
- Exception objects carry useful attributes (args, errno, filename...).

7. What's Next

Beyond catching: how to raise exceptions, and use the else/finally clauses.

Lesson 3: raise, else, finally

Module: 8 — Error Handling

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- raise exceptions to signal problems.
- Use else and finally with try/except.
- Re-raise an exception after partial handling.

Prerequisites

- Module 8 Lesson 2

1. Why This Matters

Catching errors is half the picture. Knowing when to throw an error — and how to guarantee cleanup runs — is the other half. These pieces let you write functions that fail loudly when their contract is broken.

2. Concept

2.1 raise

```
raise ValueError("age must be non-negative")
```

You pick the type and the message. Raising stops execution immediately; the nearest matching except (up the call stack) handles it, or the program crashes.

2.2 When to raise

- Input violates the function's contract ($\text{age} < 0$).
- An invariant is broken (balance < 0 after a withdraw).
- A required resource is missing.

Don't raise as a substitute for control flow — raise is for exceptional conditions.

2.3 Re-raising

After logging or partial handling, re-raise with bare raise:

```
try:
    risky()
except Exception:
    log("failed")
    raise                # passes the same exception up
```

2.4 else clause

Runs if no exception happened in the try block:

```

try:
    f = open("data.txt")
except FileNotFoundError:
    print("no file")
else:
    print("got file, now reading")
    data = f.read()
    f.close()

```

Keeps the try block focused on the operation that can fail.

2.5 finally clause

Runs always, whether an exception happened or not:

```

try:
    f = open("data.txt")
    do_stuff(f)
finally:
    f.close()          # always runs

```

with makes most finally uses unnecessary — but finally is still useful for non-file cleanup (network connections, locks, temp dirs).

2.6 All together

```

try:
    operation()
except SpecificError as e:
    handle(e)
else:
    on_success()
finally:
    cleanup()

```

2.7 Raise with from for chaining

```

try:
    int(value)
except ValueError as e:
    raise RuntimeError("config corrupted") from e

```

The traceback shows both — useful when wrapping a low-level error in a higher-level one.

3. Code Examples

Example 1: Validate with raise

```

def set_age(age):
    if not isinstance(age, int):
        raise TypeError(f"age must be int, got {type(age).__name__}")
    if age < 0:
        raise ValueError("age cannot be negative")
    print("age set to", age)

try:

```

```
    set_age(-1)
except ValueError as e:
    print("Caught:", e)
```

Expected Output: Caught: age cannot be negative

Example 2: else clause

```
try:
    n = int("42")
except ValueError:
    print("bad number")
else:
    print("parsed:", n)
```

Expected Output:

```
parsed: 42
```

Example 3: finally cleanup

```
def do_work():
    print("opening")
    try:
        print("working")
        raise RuntimeError("kaboom")
    finally:
        print("closing")

try:
    do_work()
except RuntimeError as e:
    print("caught:", e)
```

Expected Output:

```
opening
working
closing
caught: kaboom
```

Explanation: finally ran even though the exception propagated.

Example 4: Re-raising

```
def load(path):
    try:
        return open(path).read()
    except OSError:
        print(f"[log] failed to load {path}")
        raise

try:
    load("nope.txt")
except OSError as e:
    print("handled at top:", e)
```

Expected Output:

```
[log] failed to load nope.txt  
handled at top: [Errno 2] No such file or directory: 'nope.txt'
```

Example 5: Chained exception

```
def parse_config(raw):  
    try:  
        return int(raw)  
    except ValueError as e:  
        raise RuntimeError("invalid config") from e  
  
try:  
    parse_config("abc")  
except RuntimeError as e:  
    print("Caught:", e)  
    print("Cause:", e.__cause__)
```

Expected Output:

```
Caught: invalid config  
Cause: invalid literal for int() with base 10: 'abc'
```

4. Common Mistakes

Mistake	What Happens	Fix
raise "string"	TypeError — must raise an exception instance	raise ValueError("msg")
Catching just to print + ignore	Bug disappears	Re-raise with raise
finally returning a value	Overrides try/except return — confusing	Don't return in finally
else always runs (it doesn't)	Confusion	else runs only if no exception

5. Practice Tasks

- Write `safe_int(s)` that returns `int(s)` or raises `ValueError("not an int: '...') with the bad value in the message.`
- Wrap a file-read with `try/except/else/finally`: print "ok" in `else`, "done" in `finally`.
- Wrap `int(s)` in `try/except`, catch `ValueError`, log it, and re-raise.

6. Key Takeaways

- `raise ExceptionType("msg")` signals errors deliberately.
- `else` runs when `try` succeeds; `finally` always runs.
- Re-raise with bare `raise` to propagate after logging.
- Use `from` to chain related exceptions.

7. What's Next

How to define your own exception classes — a small peek at OOP before Module 9.

Lesson 4: Custom Exceptions

Module: 8 — Error Handling

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Define a custom exception class.
- Add fields and useful messages.
- Use a hierarchy of custom exceptions for a module.

Prerequisites

- Module 8 Lesson 3

1. Why This Matters

Built-in exceptions are great, but sometimes you want `InsufficientFundsError`, not `ValueError`. Custom exceptions make your code self-documenting and let callers catch your domain errors precisely.

This lesson is also a tiny preview of OOP (Module 9). You only need 2 lines to define a useful exception class.

2. Concept

2.1 Smallest custom exception

```
class MyError(Exception):  
    pass
```

That's it. Use it:

```
raise MyError("something specific went wrong")
```

The class `... (Exception):` says "this is a kind of `Exception`" — inheritance. `pass` means "no extra behavior."

2.2 Adding fields

For richer errors:

```
class InsufficientFundsError(Exception):  
    def __init__(self, balance, attempted):  
        self.balance = balance  
        self.attempted = attempted  
        super().__init__(  
            f"Tried to withdraw {attempted}, but balance is only {balance}"  
        )
```

(`__init__` is a constructor — fully covered in Module 9.)

2.3 Hierarchy

For a small library, give yourself a root + branches:

```
class BankError(Exception):
    """Base for all bank errors."""

class InsufficientFundsError(BankError): ...
class AccountClosedError(BankError): ...
class InvalidAmountError(BankError): ...
```

Callers can catch BankError for any banking error, or a specific subclass.

2.4 When NOT to make custom exceptions

- One-off scripts — ValueError is fine.
- When a stdlib exception is already a perfect fit.

Only create custom ones when they add clarity in code that others (or future-you) will read.

3. Code Examples

Example 1: Minimal custom exception

```
class TooHot(Exception):
    pass

def cook(temp):
    if temp > 200:
        raise TooHot(f"oven at {temp} is too hot")
    print("cooking at", temp)

try:
    cook(250)
except TooHot as e:
    print("Caught:", e)
```

Expected Output: Caught: oven at 250 is too hot

Example 2: Exception with data

```
class InsufficientFundsError(Exception):
    def __init__(self, balance, attempted):
        self.balance = balance
        self.attempted = attempted
        super().__init__(
            f"Tried {attempted}, only have {balance}"
        )

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(balance, amount)
    return balance - amount

try:
```

```

        withdraw(100, 500)
    except InsufficientFundsError as e:
        print(e)
        print("Short by", e.attempted - e.balance)

```

Expected Output:

```

Tried 500, only have 100
Short by 400

```

Example 3: Hierarchy

```

class BankError(Exception):
    pass

class Overdraft(BankError):
    pass

class AccountClosed(BankError):
    pass

def withdraw(account, amount):
    if account["closed"]:
        raise AccountClosed("account is closed")
    if amount > account["balance"]:
        raise Overdraft("not enough money")
    account["balance"] -= amount

acct = {"balance": 50, "closed": False}
for amt in [10, 999]:
    try:
        withdraw(acct, amt)
        print(f"withdrew {amt}, balance {acct['balance']}")
    except BankError as e:
        print("bank error:", e)

```

Expected Output:

```

withdrew 10, balance 40
bank error: not enough money

```

4. Common Mistakes

Mistake	What Happens	Fix
Inheriting from BaseException	Catches KeyboardInterrupt etc.	Inherit from Exception
Custom exception with cryptic message	Caller can't debug	Build a useful default message
Custom exception just for one place	Overkill	Use a built-in
Forgetting super().__init__(msg)	print(e) is empty	Call super or set message explicitly

5. Practice Tasks

- Define class `NegativeAmount(Exception)`: pass. Raise it from a deposit function when `amount < 0`.
- Define a class `GameError(Exception)` and one subclass class `InvalidMove(GameError)`. Raise/catch each.
- Add a code attribute to a custom exception you raise.

6. Key Takeaways

- `class MyError(Exception)`: pass is the minimum.
- Use `__init__` to add fields; call `super().__init__(msg)` for a message.
- Build a hierarchy when you have several related errors.
- Inherit from `Exception`, never `BaseException`.

7. What's Next

The last lesson — debugging techniques for when things go wrong.

Lesson 5: Debugging Techniques

Module: 8 — Error Handling

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Read a Python traceback bottom-up.
- Use `print()` debugging effectively.
- Use the `breakpoint()` / `pdb` debugger.
- Use `assert` for assumptions.

Prerequisites

- Module 8 Lesson 4

1. Why This Matters

You'll spend more time debugging than writing new code. Two skills matter most: reading tracebacks accurately and isolating the failing line. Everything else is a refinement on those.

2. Concept

2.1 Reading a traceback

```
Traceback (most recent call last):
  File "app.py", line 12, in <module>
    main()
  File "app.py", line 8, in main
    print(divide(10, 0))
  File "app.py", line 4, in divide
    return a / b
ZeroDivisionError: division by zero
```

Read bottom-up:

- Last line — the exception type and message.
- Frame above — the line that actually failed (`return a / b`).
- Above that — how you got there (function call chain).

2.2 print debugging — done well

Sprinkle prints. Label them. Show the variable name with `=`:

```
print(f"{user=}, {balance=}, {amount=}")
```

Use a unique tag (`"[DBG]"`) so you can `grep` later.

Remove or comment out when done — leftover prints clutter output.

2.3 breakpoint()

Insert breakpoint() to drop into the debugger at that line:

```
def withdraw(balance, amount):  
    breakpoint()  
    return balance - amount
```

In the debugger prompt:

Command	Action
n	next line
s	step into function
c	continue
p var	print a variable
pp var	pretty-print
l	list source around current
q	quit
h	help

Exit by typing c to continue (or q to abort).

2.4 VS Code debugger

Set a breakpoint by clicking the gutter; F5 to run; hover variables. We covered this in Module 1 Lesson 6.

2.5 assert

assert condition, "message" raises AssertionError if condition is false:

```
def average(nums):  
    assert len(nums) > 0, "average() needs at least one number"  
    return sum(nums) / len(nums)
```

Use asserts to enforce invariants you believe must be true. Don't use them for user-input validation — Python can disable asserts with python -O. For user input, raise a real exception.

2.6 Strategy when stuck

- Read the traceback to find the failing line.
- Print the variables on that line.
- If unclear, drop a breakpoint() just before it.
- Check assumptions: what type is each value? what's the actual value?
- Form a hypothesis. Make the smallest change that tests it.
- Repeat. Most bugs come from a wrong assumption you didn't notice.

2.7 Use the right tool for the symptom

Symptom	Tool
Hard crash with traceback	Read traceback, fix the line
Wrong output but no crash	Add prints to narrow down

Random failure	breakpoint at the failure
Need to inspect deep state	VS Code debugger or pdb
Theory you want to verify	assert

3. Code Examples

Example 1: Read a traceback

```
def divide(a, b):
    return a / b

def main():
    print(divide(10, 0))

main()
```

Expected Output:

```
Traceback (most recent call last):
  File "trace.py", line 7, in <module>
    main()
  File "trace.py", line 5, in main
    print(divide(10, 0))
  File "trace.py", line 2, in divide
    return a / b
ZeroDivisionError: division by zero
```

Example 2: Debug print

```
def total(items):
    print(f"[DBG] {items=}")
    s = 0
    for it in items:
        print(f"[DBG] adding {it=}, running {s=}")
        s += it
    return s

print(total([3, 1, 4]))
```

Expected Output:

```
[DBG] items=[3, 1, 4]
[DBG] adding it=3, running s=0
[DBG] adding it=1, running s=3
[DBG] adding it=4, running s=4
8
```

Example 3: breakpoint

```
def withdraw(balance, amount):
    # breakpoint()          # uncomment to enter pdb
    return balance - amount

print(withdraw(100, 30))
```

Expected Output: 70

(If you uncomment the breakpoint, you'll get an interactive prompt.)

Example 4: assert

```
def average(nums):
    assert len(nums) > 0, "average() needs at least one number"
    return sum(nums) / len(nums)

print(average([3, 1, 4]))
try:
    print(average([]))
except AssertionError as e:
    print("AssertionError:", e)
```

Expected Output:

```
2.6666666666666665
AssertionError: average() needs at least one number
```

4. Common Mistakes

Mistake	What Happens	Fix
Reading traceback top-down	Misses the real cause	Read bottom-up
Using assert for user input	Disabled by -O; user gets a crash	Use raise ValueError(...)
Print scatter without labels	Output is meaningless	Use {var=} and tags
Changing two things at once when debugging	Don't know what fixed it	One change at a time

5. Practice Tasks

- Cause and read a KeyError traceback — write down what each line tells you.
- Add 3 debug prints to a buggy function, then remove them once fixed.
- Add an assert in a function and trigger it.

6. Key Takeaways

- Read tracebacks bottom-up.
- Print, print, print — but label and remove.
- breakpoint() drops into the debugger; n/p/c/q get you far.
- assert documents invariants — not for user input.

7. What's Next

End of Module 8. Next: Module 9 — Object-Oriented Programming, where we learn how to define classes (a core skill, already previewed in custom exceptions).

Module 8 — Exercises

15 exercises.

8.1 try/except

- (E) Wrap `int(input())` in try/except; catch `ValueError`, print "not a number".
- (M) Wrap `open("missing.txt")` in try/except; print a clean error.
- (H) Loop until the user enters a valid float; catch `ValueError`.

8.2 Hierarchy

- (E) Trigger and catch `KeyError`, `IndexError`, `TypeError` — print type names.
- (M) Use except `LookupError` to catch both dict and list lookups in one block.
- (H) Inspect `e.errno`, `e.strerror` on a `FileNotFoundError`.

8.3 raise/else/finally

- (E) Write `set_age(age)` that raises `ValueError` if negative.
- (M) Use `else` to print "ok" after a successful `int()` conversion, in a try/except.
- (H) Build try/except/else/finally that opens a file, prints success in `else`, "done" in `finally`.

8.4 Custom exceptions

- (E) Define class `InvalidEmail(Exception)`: pass and raise it from a `register(email)` if "@" not in email.
- (M) Add a code attribute to a custom exception and read it in the except.
- (H) Build a class `GameError(Exception)` with two subclasses; raise each in different conditions; catch both with `GameError`.

8.5 Debugging

- (E) Cause a `ZeroDivisionError` in a 3-level function call. Read the traceback bottom-up; write down each line.
- (M) Use `print(f'{x=}')` to debug a faulty sum function.
- (H) Add `breakpoint()` to a function and step through one call using `n` and `p` var.

Module 8 — Quiz

10 MCQs.

Q1

Which is the best exception type to catch for `int("hello")`? A. `TypeError` B. `ValueError` C. `Exception` D. `RuntimeError`

Answer: B (L2)

Q2

Which block runs whether an exception happened or not? A. `except` B. `else` C. `finally` D. `raise`

Answer: C (L3)

Q3

When does the `else` clause of `try/except` run? A. always B. only when an exception happened C. only when no exception happened D. when `finally` ran

Answer: C (L3)

Q4

Which is NOT a good practice? A. Catching the narrowest type B. Re-raising after logging with bare `raise` C. Bare `except:` D. `except Exception as e:`

Answer: C (L1)

Q5

What's the root of "stuff you should normally catch"? A. `BaseException` B. `Exception` C. `Error` D. `Throwable`

Answer: B (L2)

Q6

What's wrong with `raise "bad"`? A. Nothing B. You can only raise exception instances, not strings C. Missing parens D. Strings auto-convert

Answer: B (L3)

Q7

The simplest custom exception: A. `class E: pass` B. `class E(Exception): pass` C. `class E(BaseException): pass` D. `def E(): raise`

Answer: B (L4)

Q8

What does breakpoint() do? A. Stops the program B. Drops into the Python debugger at that line
C. Logs to a file D. Skips the next line

Answer: B (L5)

Q9

Which is true about assert? A. Should be used to validate user input B. Can be disabled with
python -O C. Always raises Exception D. Stops the interpreter even when caught

Answer: B (L5)

Q10

Read this traceback. Which line failed?

```
Traceback (most recent call last):
  File "a.py", line 10, in <module>
    main()
  File "a.py", line 7, in main
    work()
  File "a.py", line 3, in work
    return 1 / 0
ZeroDivisionError: division by zero
```

A. line 10 B. line 7 C. line 3 D. all of them

Answer: C — the bottom-most code frame is where the error originated. (L5)

Module 8 — Assignment: Robust Bank Account

Estimated time: 1.5 hours

Combines: Module 8 + Modules 4 + 7

Brief

Take the Module-4 banking functions and harden them with proper error handling. Persist account state to JSON between runs. Use custom exceptions for domain errors.

Requirements

- Custom exceptions:
- `BankError(Exception)` — base.
- `InvalidAmountError(BankError)` — negative amount.
- `InsufficientFundsError(BankError)` — overdraft attempt; expose `.balance` and `.attempted`.
- `AccountNotFoundError(BankError)` — unknown account.
- Functions (all in one file):
- `deposit(accounts, name, amount)` — raises on bad inputs.
- `withdraw(accounts, name, amount)` — raises on overdraft.
- `transfer(accounts, src, dst, amount)` — uses `deposit+withdraw`; rolls back if `withdraw` succeeds but `deposit` fails (use `try/except + reverse`).
- `load(path)` — reads accounts dict from JSON; returns `{}` if file missing.
- `save(accounts, path)` — writes JSON with `indent=2`.
- A `main()` that:
- Loads `accounts.json`.
- Runs a small sequence of operations, each wrapped in `try/except`, printing friendly messages.
- Saves on exit (use `try/finally` to guarantee save).

Constraints

- Modules 1-8 only.
- Use `pathlib`, `with`, `json`.
- Custom exceptions must inherit from `BankError`.
- No prints inside the operation functions — they only raise or succeed.

Expected Behavior (sample)

```
[ok] deposit 500 to alice
[err] InvalidAmountError: amount must be positive (got -10)
[ok] withdraw 100 from alice
[err] InsufficientFundsError: tried 10000, only have 400
[ok] transfer 50 from alice -> bob
Saved accounts.json
```

Grading Rubric

Criterion	Weight
Custom exceptions used	25%
All ops handle errors	25%
Persist load/save	20%
try/finally saves on exit	15%
Style + docstrings	15%

Submission

Save as assignment_module8.py.

Module 8 — Mini Project: Safer Notes Manager

Overview

Take the Module-7 Notes Manager and harden it. Every file operation, every input parse, every command dispatch should fail gracefully with a clear error message — not a traceback.

Concepts Used

- try/except, custom exceptions (M8)
- JSON + pathlib + with (M7)
- Dicts + lists (M5)

Features

- All previous Notes Manager commands.
- Custom exceptions: NotesError, NoteNotFound, BadInput.
- Wrap each command so a bad input shows a clean error and the loop continues.
- Auto-create notes.json if missing; tolerate corrupted JSON (fall back to empty).

Starter Code

```
# Mini Project: Safer Notes Manager
import json
from datetime import datetime
from pathlib import Path

NOTES_FILE = Path("notes.json")

class NotesError(Exception):
    pass

class NoteNotFound(NotesError):
    pass

class BadInput(NotesError):
    pass

def load():
    if not NOTES_FILE.exists():
        return []
    try:
        with NOTES_FILE.open(encoding="utf-8") as f:
            return json.load(f)
    except json.JSONDecodeError:
        print("[warn] notes.json was corrupted – starting fresh")
        return []

def save(notes):
    with NOTES_FILE.open("w", encoding="utf-8") as f:
        json.dump(notes, f, indent=2)
```

```

def find(notes, nid):
    for n in notes:
        if n["id"] == nid:
            return n
    raise NoteNotFound(f"no note with id {nid}")

def add(notes):
    title = input("Title: ").strip()
    if not title:
        raise BadInput("title cannot be empty")
    body = input("Body: ")
    nid = max((n["id"] for n in notes), default=0) + 1
    notes.append({
        "id": nid,
        "created": datetime.now().isoformat(timespec="seconds"),
        "title": title,
        "body": body,
    })

def parse_id(s):
    try:
        return int(s)
    except ValueError as e:
        raise BadInput(f"id must be a number, got '{s}'") from e

def run():
    notes = load()
    while True:
        try:
            cmd = input("\n[add|list|view ID|del ID|find Q|quit] > ").strip().split(maxsplit=1)
            if not cmd:
                continue
            match cmd[0].lower():
                case "add":
                    add(notes); save(notes); print("added.")
                case "list":
                    for n in notes:
                        print(f"{n['id']:>3} | {n['title']}")
                case "view":
                    if len(cmd) < 2:
                        raise BadInput("view needs an id")
                    n = find(notes, parse_id(cmd[1]))
                    print(f"\n# {n['title']}\n{n['body']}")
                case "del":
                    if len(cmd) < 2:
                        raise BadInput("del needs an id")
                    n = find(notes, parse_id(cmd[1]))
                    notes.remove(n); save(notes); print("deleted.")
                case "find":
                    if len(cmd) < 2:
                        raise BadInput("find needs a query")
                    q = cmd[1].lower()

```

```

        for n in notes:
            if q in n["title"].lower() or q in n["body"].lower():
                print(f"{n['id']:>3} | {n['title']}")
    case "quit":
        print("Bye!"); break
    case _:
        print("Unknown command.")
except NotesError as e:
    print(f"[error] {e}")
except KeyboardInterrupt:
    print("\nbye!"); break

if __name__ == "__main__":
    run()

```

Expected Interaction

```

[add|list|view ID|del ID|find Q|quit] > view
[error] view needs an id
[add|list|view ID|del ID|find Q|quit] > view abc
[error] id must be a number, got 'abc'
[add|list|view ID|del ID|find Q|quit] > view 99
[error] no note with id 99
[add|list|view ID|del ID|find Q|quit] > add
Title:
[error] title cannot be empty
[add|list|view ID|del ID|find Q|quit] > add
Title: Shopping
Body: bread, milk
added.
[add|list|view ID|del ID|find Q|quit] > quit
Bye!

```

Extension Challenges

- Add timestamps to error logging in a separate errors.log file.
- Add a BadIdFormat subclass of BadInput.
- Add a --debug flag that switches between friendly errors and full tracebacks.

Grading Rubric

Criterion	Weight
Custom exception hierarchy	25%
All commands handle errors	30%
Corrupted JSON tolerated	15%
Code stays readable	15%
Style	15%